Lab No: 05

Lab Name

Simulation of Contiguous Memory Allocation Techniques (First Fit, Best Fit, Worst Fit)

Objectives

- To understand the concept of contiguous memory allocation in Operating Systems.
- To study and implement First Fit, Best Fit, and Worst Fit memory allocation techniques.
- To compare different allocation strategies based on memory utilization and fragmentation.

Background Study (with Example)

Contiguous memory allocation is a memory management technique where each process is allocated a single continuous block of memory. When a process requests memory, the operating system searches for a suitable free block using an allocation strategy.

First Fit

The first available memory block that is large enough to satisfy the process request is allocated.

Best Fit

The smallest memory block that is sufficient for the process is allocated, reducing internal fragmentation.

Worst Fit

The largest available memory block is allocated to the process, leaving large remaining free spaces.

Example

Memory Blocks: 100KB, 500KB, 200KB, 300KB, 600KB

Process Requests: 212KB, 417KB, 112KB, 426KB

Different algorithms allocate memory differently, leading to varying fragmentation.

Code (with Results)

Start here | lab 6.c | OS LAB 5.c

```c
#include <stdio.h>

void firstFit(int blockSize[], int m, int processSize[], int n) {
    int allocation[10];
    for (int i = 0; i < n; i++) allocation[i] = -1;

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            if (blockSize[j] >= processSize[i]) {
                allocation[i] = j;
                blockSize[j] -= processSize[i];
                break;
            }
        }
    }

    printf("\nFirst Fit Allocation:\n");
    for (int i = 0; i < n; i++) {
        if (allocation[i] != -1)
            printf("Process %d allocated to Block %d\n", i + 1, allocation[i] + 1);
        else
            printf("Process %d not allocated\n", i + 1);
    }
}

void bestFit(int blockSize[], int m, int processSize[], int n) {
    int allocation[10];
    for (int i = 0; i < n; i++) allocation[i] = -1;

    for (int i = 0; i < n; i++) {
        int bestIdx = -1;
        for (int j = 0; j < m; j++) {
            if (blockSize[j] >= processSize[i]) {
                if (bestIdx == -1 || blockSize[j] < blockSize[bestIdx])
                    bestIdx = j;
            }
        }
        if (bestIdx != -1) {
            allocation[i] = bestIdx;
            blockSize[bestIdx] -= processSize[i];
        }
    }

    printf("\nBest Fit Allocation:\n");
    for (int i = 0; i < n; i++) {
        if (allocation[i] != -1)
            printf("Process %d allocated to Block %d\n", i + 1, allocation[i] + 1);
        else
            printf("Process %d not allocated\n", i + 1);
    }
}

void worstFit(int blockSize[], int m, int processSize[], int n) {
    int allocation[10];
    for (int i = 0; i < n; i++) allocation[i] = -1;

    for (int i = 0; i < n; i++) {
        int worstIdx = -1;
        for (int j = 0; j < m; j++) {
            if (blockSize[j] >= processSize[i]) {
                if (worstIdx == -1 || blockSize[j] > blockSize[worstIdx])
                    worstIdx = j;
            }
        }
        if (worstIdx != -1) {
            allocation[i] = worstIdx;
            blockSize[worstIdx] -= processSize[i];
        }
    }

    printf("\nWorst Fit Allocation:\n");
    for (int i = 0; i < n; i++) {
        if (allocation[i] != -1)
```

```c
        if (allocation[i] != -1)
            printf("Process %d allocated to Block %d\n", i + 1, allocation[i] + 1);
        else
            printf("Process %d not allocated\n", i + 1);
    }
}

int main() {
    int m, n, choice;
    int blockSize[10], processSize[10];

    printf("Enter number of memory blocks: ");
    scanf("%d", &m);
    printf("Enter size of each block:\n");
    for (int i = 0; i < m; i++) scanf("%d", &blockSize[i]);

    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter size of each process:\n");
    for (int i = 0; i < n; i++) scanf("%d", &processSize[i]);

    while (1) {
        printf("\n1. First Fit\n2. Best Fit\n3. Worst Fit\n4. Exit\nEnter choice: ");
        scanf("%d", &choice);

        int tempBlock[10];
        for (int i = 0; i < m; i++) tempBlock[i] = blockSize[i];

        switch (choice) {
            case 1: firstFit(tempBlock, m, processSize, n); break;
            case 2: bestFit(tempBlock, m, processSize, n); break;
            case 3: worstFit(tempBlock, m, processSize, n); break;
            case 4: return 0;
            default: printf("Invalid choice\n");
        }
    }
}
```

Sample Output

```
"E:\5th semester\Operating S
Enter number of memory blocks: 10
Enter size of each block:
10
10
20
10
2
3
6
5
2
5
Enter number of processes: 3
Enter size of each process:
6
8
83

1. First Fit
2. Best Fit
3. Worst Fit
4. Exit
Enter choice: 1

First Fit Allocation:
Process 1 allocated to Block 1
Process 2 allocated to Block 2
Process 3 not allocated
```

```
1. First Fit
2. Best Fit
3. Worst Fit
4. Exit
Enter choice: 4

Process returned 0 (0x0)   execution time : 38.293 s
Press any key to continue.
```

- First Fit: Some processes allocated sequentially
- Best Fit: Reduced internal fragmentation
- Worst Fit: Larger leftover memory blocks

Discussion

First Fit is fast and simple but may cause fragmentation. Best Fit reduces waste but is slower due to searching. Worst Fit attempts to leave large free blocks but is often inefficient. Among them, Best Fit generally provides better memory utilization in contiguous allocation systems.